

Tema 1 – Fundamentos de la Ingeniería del Software

Este documento no sustituye a los apuntes del tema ni a los ejercicios como tests, o actividades H5P. Es únicamente un documento útil para el repaso.

Objetivos de aprendizaje

- 1.1.** Diferenciar software a medida de software genérico, clasificar los dominios de aplicación del software y los sistemas de información como categoría destacada, y enumerar los atributos que definen un software de calidad.
- 1.2.** Identificar los principales factores que dificultan el desarrollo de software y sus consecuencias, y argumentar, a partir de casos históricos documentados, por qué los fallos de software pueden tener consecuencias humanas, económicas o reputacionales graves.
- 1.3.** Definir la Ingeniería del Software y diferenciarla de la programación individual, la Ciencia de la Computación y la Ingeniería de Sistemas, explicar el papel del modelado como actividad central de la disciplina, e identificar las principales guías y estándares de referencia (SWEBOK, SE2014, ISO/IEC/IEEE 29148, UML).
- 1.4.** Describir los roles profesionales habituales en un equipo de desarrollo de software, distinguir la ruta técnica de la ruta de gestión en la carrera del ingeniero de software, y aplicar los principios del Código de Ética de ACM/IEEE-CS al ejercicio de la profesión.

Esenciales por bloque

1.1 · El software: evolución y características

- El software cumple una doble función: es un producto que satisface necesidades concretas y se comercializa, y también un vehículo de transmisión de información y conocimiento.
- No se reduce al código: incluye programas, archivos de configuración, documentación técnica, manuales de instalación y uso, y otros elementos de soporte (sitios web, actualizaciones).
- El software es un elemento lógico: su coste procede del tiempo de desarrollo y mantenimiento, no de materiales. No se desgasta físicamente, pero "se deteriora" con el tiempo por acumulación de deuda técnica (código de baja calidad, documentación desactualizada o inexistente).
- La crisis del software (finales de los años 60): plazos incumplidos, presupuestos desbordados, sistemas inmantenibles. Puso de manifiesto la necesidad de un enfoque sistemático y motivó el nacimiento de la Ingeniería del Software como disciplina.
- Software a medida (desarrollado para un cliente concreto): mayor control del proceso y flexibilidad, menor coste inicial, pero mayor coste total y tiempo de desarrollo. Software genérico (producido para un mercado amplio): menor coste total por economía de escala y menor tiempo de desarrollo por reutilización, pero menor personalización y flexibilidad. Ejemplos de software genérico: Photoshop, Office, Notion, Salesforce, Kaspersky.
- Dominios de aplicación del software (clasificación no exhaustiva): software de sistema, software de aplicación, software científico o de ingeniería, software integrado (embebido), software de línea de productos, aplicaciones web/móviles, software de inteligencia artificial.
- Sistemas de información: recogen, almacenan, procesan y distribuyen información para apoyar la toma de decisiones (ejemplos: ERP, CRM, DSS). Se caracterizan por alta fiabilidad, uso intensivo de datos, multiplicidad de usuarios concurrentes, alta frecuencia de cambio (reglas de negocio, legislación) e integración con otros sistemas; representan una proporción muy significativa de los proyectos de desarrollo profesional.
- Software heredado (legacy software): sistemas antiguos y a menudo monolíticos (es decir, no modulares), con documentación limitada, que siguen en uso pese a sus problemas de seguridad y escalabilidad. Razones habituales para mantenerlo:
 - Funcionalidad comprobada: lleva tiempo demostrando ser fiable para el uso al que se destina.
 - Costes de reemplazo: migración de datos, nuevas funcionalidades y capacitación del personal.

- Integración con la infraestructura existente: sustituirlo puede exigir reconstruir procesos críticos ya conectados a él.
- Experiencia y conocimiento del personal: el equipo domina un sistema cuyo reemplazo implicaría una curva de aprendizaje.
- Cumplimiento normativo y regulatorio: el sistema puede estar diseñado a medida de requisitos legales concretos.
- Atributos del software de calidad: funcionalidad, mantenibilidad, confiabilidad, eficiencia y usabilidad. La calidad es multidimensional: no se reduce a cumplir los requisitos funcionales, y su peso relativo depende del tipo de aplicación y de las prioridades del proyecto.

1.2 · Factores que afectan al desarrollo de software

- Principales causas de los problemas en el desarrollo:
 - Complejidad inherente de los sistemas modernos.
 - Falta de un enfoque estructurado: metodologías inadecuadas o inexistentes.
 - Problemas de comunicación entre equipos y con los usuarios.
 - Seguridad descuidada.
 - Dificultades de mantenimiento.
 - Captación y rotación de talento, incluida la falta de diversidad de género o étnica en los equipos, que puede limitar la creatividad y la innovación.
 - Estimaciones y planificaciones imprecisas.
- Consecuencias más frecuentes: retrasos en el lanzamiento, sobrecostes, deuda técnica, incumplimiento de requisitos, pérdida de datos, vulnerabilidades de seguridad, insatisfacción y pérdida de confianza del cliente, pérdida de ingresos, costes adicionales e impacto negativo en la productividad.
- Incluso tras pruebas exhaustivas, los grandes sistemas conservan errores no detectados, por causas como la complejidad del sistema, la diversidad de plataformas y dispositivos, los cambios constantes, los plazos ajustados y las dificultades de comunicación entre los actores del desarrollo.
- Casos históricos que ilustran el coste de los fallos de software: **Therac-25** (1985, dispositivo de radioterapia; variable no inicializada en el código; dos pacientes fallecidos por sobredosis de radiación). **Mars Climate Orbiter** (1999, NASA; pérdida de la sonda por una discrepancia entre unidades métricas e imperiales no detectada). **Ariane 5** (1996, ESA; desbordamiento de un valor representado en 16 bits al reutilizar sin adaptar el software del Ariane 4). **Heartbleed** (2014; vulnerabilidad en OpenSSL que expuso información confidencial en millones de servidores durante dos años). **Knight Capital** (2012; error en una actualización del software de trading que provocó pérdidas de 440

millones de dólares en menos de una hora y la quiebra de la empresa en dos días). **Boeing 737 MAX** (2018-2019; fallo del sistema MCAS por depender de un único sensor de ángulo de ataque; 346 víctimas mortales).

1.3 · Conceptos clave y definición de la Ingeniería del Software

- La Ingeniería del Software surge como disciplina formal en 1968, en una conferencia de la OTAN organizada por Friedrich L. Bauer, como respuesta a la crisis del software. Margaret Hamilton (MIT) es una figura pionera de la época, por su trabajo en el software de navegación de la misión Apolo.
- Definiciones de referencia. Pressman: "establecimiento y uso de principios de ingeniería robustos, orientados a obtener software económico, fiable, eficiente y que satisfaga las necesidades del usuario". Sommerville: "disciplina de ingeniería que comprende todos los aspectos de la producción de software, desde las etapas iniciales de la especificación del sistema, hasta el mantenimiento de este".
- Objetivos de la Ingeniería del Software: mejorar la calidad y la eficiencia del software, reducir costes y tiempos de desarrollo y mejorar la productividad, aplicar principios y buenas prácticas, abordar la complejidad del software, promover la estandarización, favorecer la documentación y mejorar la satisfacción del usuario.
- El método de resolución de problemas de George Pólya (1945, *How to Solve It*) — comprender el problema, diseñar un plan, ejecutar el plan, revisar el resultado— es un paralelismo útil con el ciclo de vida del software (requisitos, diseño, implementación, verificación/validación); en ambos casos el proceso es iterativo, no lineal.
- La Ingeniería del Software se diferencia de la programación individual (por su escala, planificación, gestión de proyecto y exigencia de calidad no opcional), de la Ciencia de la Computación (estudio teórico de la computación frente a la aplicación práctica de esos principios) y de la Ingeniería de Sistemas (que aborda el sistema completo —software, hardware, redes— frente al software como componente de ese sistema).
- El modelado es una actividad central de la disciplina: los modelos del dominio del problema (análisis de requisitos) y los modelos del dominio de la solución (diseño del sistema) permiten comprender y comunicar el sistema antes de construirlo. UML es el lenguaje de modelado estándar (diagramas de clases, de flujo de datos, de secuencia, de casos de uso, entre otros).
- Guías y estándares de referencia: **SWEBOK** (Software Engineering Body of Knowledge, IEEE Computer Society), **SE2014/SEEK** (guías curriculares conjuntas de IEEE-CS y ACM), **ISO/IEC/IEEE 29148:2018** (ingeniería de requisitos) y **UML 2.5.1** (ISO/IEC 19501, desarrollado por el OMG).

1.4 · El/la ingeniero/a de software: rol, responsabilidades y aspectos éticos

- El trabajo del ingeniero de software cubre todo el ciclo de vida del desarrollo (planificación, análisis de requisitos, diseño, implementación, pruebas, mantenimiento), aplicando herramientas, metodologías, procesos, y marcos de trabajo como Scrum, Kanban o DevOps.
- Roles especializados habituales:
 - Desarrollo: desarrollador frontend, desarrollador backend, arquitecto de software.
 - Datos e infraestructura: ingeniero DevOps, ingeniero de datos, analista de datos, ingeniero de infraestructura en la nube.
 - Diseño y requisitos: diseñador UI/UX, analista funcional.
 - Gestión: director de proyecto, product owner, director de producto.
 - Calidad y seguridad: ingeniero de seguridad, ingeniero de pruebas de software (QA).
 - Roles emergentes: ingeniero de machine learning, ingeniero de realidad aumentada y virtual (AR/VR), especialista en blockchain, ingeniero de accesibilidad.
- Carrera profesional (Gergely Orosz, *The Software Engineer's Guidebook*): dos rutas complementarias, no excluyentes. Ruta técnica: Software Engineer → Senior Engineer → Staff Engineer → Senior Staff Engineer → Principal Engineer → Distinguished Engineer → Fellow. Ruta de gestión: Manager → Director → Senior Director → VP of Engineering → Senior VP of Engineering → CTO. Es habitual transitar de una ruta a la otra a lo largo de la carrera.
- Código de Ética Profesional (ACM + IEEE-CS), ocho principios: interés público, cliente y empleador, producto, juicio profesional, gestión, profesión, colegas y automejora. La ética no es un complemento: es un componente esencial de la práctica profesional, porque cualquier decisión técnica puede tener implicaciones sociales, económicas o medioambientales.

Tabla de decisión: ¿software a medida o software genérico?

Situación del proyecto	Elección recomendada	Por qué
Requisitos muy específicos de una organización, sin equivalente satisfactorio en el mercado	Software a medida	Solo el desarrollo a medida puede ajustarse con precisión a necesidades y procesos particulares
Necesidad de integración estrecha con sistemas y procesos ya existentes	Software a medida	El mayor control sobre el proceso de desarrollo permite adaptar el software a la infraestructura existente
Presupuesto ajustado y necesidad de disponibilidad inmediata	Software genérico	Aprovecha la reutilización de código y la economía de escala; menor tiempo y coste de desarrollo
Funcionalidad común a muchas organizaciones del mismo sector (por ejemplo, ofimática o gestión de contactos)	Software genérico	No justifica el coste y el tiempo adicionales de un desarrollo a medida que ya resuelve el mercado con software genérico
Previsión de cambios frecuentes en los requisitos a medio plazo	Software a medida	Mayor flexibilidad para modificar y ampliar el sistema sin depender de las decisiones de un proveedor externo
Recursos internos limitados para supervisar un desarrollo personalizado	Software genérico	Exige menos control y seguimiento del proceso de desarrollo por parte del cliente

Nota: esta tabla es una simplificación útil a efectos pedagógicos. La elección entre software a medida y software genérico debe tener en cuenta múltiples aspectos del proyecto, y en la práctica muchas organizaciones combinan ambas opciones (por ejemplo, un producto genérico con módulos desarrollados a medida), por lo que la decisión rara vez se reduce a aplicar una sola fila de la tabla de forma aislada.

Confusiones habituales

- **Ingeniería del Software vs. Ciencia de la Computación** — la Ciencia de la Computación estudia los fundamentos teóricos de la computación; la Ingeniería del Software aplica esos fundamentos para producir sistemas de software de forma sistemática y con garantías de calidad.

- **Ingeniería del Software vs. Ingeniería de Sistemas** — la Ingeniería de Sistemas aborda el sistema completo (software, hardware, redes y su interacción con el entorno); la Ingeniería del Software se centra en el software como componente de ese sistema.
- **Ingeniería del Software vs. programación individual** — programar bien no equivale a aplicar ingeniería de software: esta añade planificación, gestión del proyecto, control de calidad y consideración de objetivos económicos, imprescindibles a partir de cierta escala y complejidad.
- **Software genérico vs. software a medida** — el genérico no es "peor" ni siempre "más barato": su menor coste depende de la economía de escala, mientras que el software a medida implica mayor coste total pero menor coste inicial y mayor flexibilidad.
- **Modelo del dominio del problema vs. modelo del dominio de la solución** — el primero (análisis) describe qué necesita el usuario; el segundo (diseño) describe cómo se construirá el sistema. No deben confundirse ni fusionarse.
- **Deuda técnica vs. error de software** — la deuda técnica es una decisión (consciente o acumulada) que compromete la calidad futura del sistema; un error de software es un fallo puntual de implementación. Pueden estar relacionados, pero no son lo mismo.
- **SWEBOK vs. SE2014** — el SWEBOK describe el conocimiento de la disciplina, es decir, qué debe saber un profesional en ejercicio; el SE2014/SEEK define el currículo de formación, es decir, qué debe enseñarse en un grado.
- **Software heredado (legacy) vs. software simplemente obsoleto** — un sistema heredado sigue en uso porque aporta valor (funcionalidad probada, integración, cumplimiento normativo); estar desactualizado no implica necesariamente que deba sustituirse.
- **Ruta técnica vs. ruta de gestión** — no son fases sucesivas de una misma carrera ni una es "superior" a la otra: son dos trayectorias profesionales paralelas, y es habitual transitar entre ellas a lo largo de la vida laboral.

Glosario mínimo

Ingeniería del Software: disciplina de ingeniería que aplica principios sistemáticos al desarrollo, la explotación y el mantenimiento del software, orientada a obtener software de calidad, fiable y eficiente.

Software a medida: software desarrollado específicamente para satisfacer los requisitos de un cliente concreto.

Software genérico: software producido por una organización y ofrecido en el mercado para su compra y uso por cualquier cliente.

Sistema de información: software que recoge, almacena, procesa y distribuye información para apoyar la toma de decisiones y la operativa de una organización.

Software heredado (legacy software): sistema desarrollado en el pasado que sigue en uso por su valor operativo, pese a estar técnicamente desactualizado.

Deuda técnica: coste futuro derivado de decisiones técnicas tomadas para resolver una necesidad inmediata a costa de la calidad o la mantenibilidad a largo plazo.

Crisis del software: conjunto de problemas de plazos, costes y calidad que, a finales de los años 60, evidenció la necesidad de un enfoque sistemático al desarrollo de software.

Atributos de calidad del software: propiedades —funcionalidad, mantenibilidad, confiabilidad, eficiencia, usabilidad— que determinan la calidad de un producto software.

SWEBOK: Software Engineering Body of Knowledge; guía de la IEEE Computer Society que describe las áreas de conocimiento de la disciplina.

SE2014 / SEEK: guías curriculares conjuntas de IEEE-CS y ACM que definen qué debe conocer un graduado en Ingeniería del Software.

UML: Unified Modeling Language; lenguaje de modelado estándar (ISO/IEC 19501) para representar gráficamente la estructura y el comportamiento de un sistema software.

Modelo del dominio del problema: representación del ámbito y los requisitos que debe resolver el sistema, obtenida mediante el análisis.

Modelo del dominio de la solución: representación de la arquitectura y los componentes del sistema que se va a construir, obtenida mediante el diseño.

Método de Pólya: enfoque estructurado de resolución de problemas en cuatro fases (comprender el problema, diseñar un plan, ejecutar el plan, revisar el resultado), propuesto por George Pólya en 1945.

Ruta técnica: trayectoria profesional del ingeniero de software centrada en la profundización técnica (de Software Engineer a Fellow).

Código de Ética (ACM/IEEE-CS): conjunto de ocho principios que orientan el comportamiento ético en el ejercicio de la Ingeniería del Software.

Ruta de gestión: trayectoria profesional del ingeniero de software centrada en el liderazgo de personas y proyectos (de Manager a CTO).

Sobre el uso de Inteligencia Artificial en la elaboración de este material

En la elaboración de esta hoja de esenciales y del mapa conceptual que la acompaña se ha utilizado un asistente de Inteligencia Artificial generativa (Claude, Anthropic), en el nivel 3 ("AI Collaboration") de la Escala AIAS (Perkins, Furze, Roe y MacVaugh, 2024), con el siguiente alcance:

- Síntesis, reestructuración y condensación del contenido del Tema 1 —ya redactado y revisado previamente por el profesor— en un formato de repaso: objetivos de aprendizaje, esenciales por bloque, tabla de decisión, confusiones habituales, glosario y mapa conceptual.
- La tabla de decisión sobre software a medida frente a software genérico es una elaboración derivada de los apartados de ventajas y desventajas de los apuntes; no es una tabla preexistente en ellos.
- No se ha incorporado contenido ajeno al texto de referencia ni conocimiento externo no solicitado.

Cualquier duda sobre el alcance de este uso puede consultarse directamente con el profesor.

Mapa conceptual

